



PAKE Combiners and Efficient Post-quantum Instantiations

Julia Hesse and Michael Rosenberg

IBM Research Europe – Zurich, Rüschlikon, Switzerland
jhs@zurich.ibm.com, research@mrosenberg.pub

Abstract. Much work has been done recently on developing password-authenticated key exchange (PAKE) mechanisms with post-quantum security. However, modern guidance recommends the use of *hybrid* schemes—schemes which rely on the combined hardness of a post-quantum assumption, e.g., Learning with Errors (LWE), and a more traditional assumption, e.g., decisional Diffie-Hellman. To date, there is no known hybrid PAKE construction, let alone a **general method** for achieving such.

In this paper, we present two efficient PAKE combiners—algorithms that take two PAKEs satisfying mild assumptions, and output a third PAKE with combined security properties—and prove these combiners secure in the Universal Composability (UC) model. Our sequential combiner, instantiated with efficient existing PAKEs such as CSpace (built on Diffie-Hellman-type assumptions) and CAKE (built on lattice assumptions), yields the first known hybrid PAKE.

Keywords: key agreement · password-based cryptography · post-quantum cryptography

1 Introduction

Memorable passwords are the one of most commonly used forms of authentication today, and exist across a wide variety of applications. While some applications, such as website login, have users send their plaintext password, many large-scale applications utilize password-authenticated key exchange (PAKE) and its variants to limit the amount of information that can be intercepted. These include iCloud Keychain escrow, 1Password user authentication, Facebook Messenger chat history sharing, WhatsApp backup encryption, and passport chip access control [1].

As a result of the widespread push from industry and government to move towards post-quantum cryptography, many efficient lattice- and isogeny-based PAKEs have been developed in recent years [3, 8, 10, 13, 23, 37, 46]. However, these

J. Hesse—The author was supported by the Swiss National Science Foundation (SNSF) under the AMBIZIONE grant “Cryptographic Protocols for Human Authentication and the IoT.”.

constructions are not necessarily the end goal of the post-quantum efforts. The largest rollout of a post-quantum protocol to date is that of TLS 1.3 with *hybrid* key exchange [7], i.e., key exchange which enjoys the protection of both newer post-quantum constructions and classical constructions. The purpose of the hybrid in this application is two-fold. Firstly, the newer, less tested post-quantum hardness assumptions may turn out to be easier than expected. Secondly, the newer implementations of the post-quantum algorithms may turn out to have exploitable bugs. Perhaps unintuitively, the second is considered by some as the more likely scenario [55], and in fact already has occurred [15, 30, 31, 36]. For these reasons, some governments recommend the use of hybrid constructions for secure protocols [25, 28]. Unfortunately, no hybrid PAKE currently exists.

1.1 Non-solutions

The most common way to develop a primitive from a hybrid assumption is to simply *combine* two copies of the primitive, with each copy relying on a different assumption. However, the obvious combiners from other cryptographic primitives do not work for PAKEs.

Concatenation Combiner. Hybrid KEMs and hybrid digital signature schemes can be constructed from the *concatenation combiner*: separately execute two copies of the primitive, and output the hash or concatenation of the outputs, respectively [11, 17]. Let us attempt this for PAKE: let P be the PAKE given by running some PAKE P_1 and another PAKE P_2 in parallel using the same password pw and then outputting the hash $H(K_1, K_2)$, where K_i is the session key computed in P_i . In general, this straightforward combiner is only guaranteed to be as strong as the *weaker* of P_1, P_2 . Intuitively, the reason is that we made the situation worse by trusting *two* PAKEs with protecting the password. For a concrete example, consider P_1 to be KC-SPAKE2 [54] and P_2 to be MLWE-PAKE [52]. In the design of KC-SPAKE2, the responder sends a MAC tag derived from their password-dependent Diffie-Hellman shared secret. An attacker with a discrete-logarithm oracle (e.g., from a quantum computer) can break P by merely observing one transcript of the KC-SPAKE2 subprotocol, and mount an offline attack where guesses are confirmed against the MAC tag. MLWE-PAKE, on the other hand, sends samples from the Module Learning with Errors (MLWE) distribution, blinded with the hashed password. An attacker with a decision-MLWE oracle who wishes to break P can observe one transcript of the MLWE-PAKE subprotocol to mount an offline attack, because the MLWE oracle will reject with overwhelming probability when the password guess is wrong. Hence, *both* assumptions have to hold in order for the concatenation combiner to protect the password.

KEM-Based PAKEs. Another potential method to develop a hybrid PAKE is to use an existing KEM-based PAKE construction, e.g., CAKE [13], and instantiate it with a hybrid KEM. Similar to the above, this does not necessarily have the combined security of the two underlying KEMs KEM_1, KEM_2 . Suppose the hybrid KEM is a concatenated KEM, as many of the existing hybrids are.

In the first communication round, CAKE encrypts the KEM public key, i.e., a concatenation of KEM_1 and KEM_2 public keys, under pw . Thus, if *either* of the underlying public keys has structure that makes it distinguishable from uniform, then trial decryption on the CAKE encrypted public key yields an offline attack. Concretely, take KEM_1 to be a DH KEM over the Ristretto prime order group [43] with points represented using the Elligator2 encoding [16], and KEM_2 to be FrodoKEM [18]. While the DH public keys are statistically indistinguishable from a uniform 32-byte string, FrodoKEM public keys are only *computationally* indistinguishable from uniform under the LWE assumption.¹ If a passive attacker possesses an LWE oracle, they may trial-decrypt the encrypted concatenated public key, ignore the DH public key, and check if the decrypted value is an LWE sample. Thus, CAKE with this hybrid KEM is at most as secure as LWE. A similar argument shows that the same is true for other recent KEM-based PAKEs such as OCAKE, CHIC, NoIC, and NICE-PAKE [8–10, 13].

1.2 Contributions

In this paper we give a generalizable method for constructing hybrid PAKEs and demonstrate efficient instantiations. We present two PAKE combiners: **ParComb**, a parallel combiner, executing both PAKEs in parallel and hashing the results; and **SeqComb**, a sequential combiner, executing the first PAKE and feeding the resulting session key into the second PAKE. We prove the combiners’ security in the universal composability (UC) model with random oracles and static corruptions.

ParComb. For our **ParComb** combiner, which follows the blueprint of the usual concatenation combiner, we need to circumvent the above issues. To this end, we formalize a property of a PAKE that is sufficient to prove **ParComb** secure. Intuitively, we demand that the password protection of both PAKEs used in **ParComb** not fatally break if any of the underlying computational assumptions break, i.e., their transcripts information-theoretically hide the password. This ensures that none of the PAKEs leak information about the input password. We identify a handful of PAKE protocols from the literature that satisfy this unconditional hiding property. Unfortunately, none of these are post-quantum PAKEs. So **ParComb**, while having zero round complexity overhead over its building blocks, does not yet yield hybrid PAKE. We conclude by pointing to concurrent work which shows that **ParComb** realizes an incomparable notion of PAKE when instantiated with a specific isogeny-based PAKE.

SeqComb. These limitations motivate looking into sequential combiner designs. Our combiner **SeqComb** derives a shared key from a password through the first PAKE, and then uses this shared key as a password in the second PAKE to derive another shared key. The final key is, again, the hashed concatenation of both PAKE keys. As in **ParComb**, the first PAKE consumes the plain password

¹ This is not particular to FrodoKEM. It appears that all standards-track lattice- and code-based KEMs achieve, at best, computational public key uniformity.

and hence must enjoy the same unconditional hiding property as both PAKEs in **ParComb**. For the second PAKE, however, we can get away with a weaker hiding property. This is because we used the first PAKE as an entropy amplifier to derive unguessable preshared keys from the passwords, and hence we do not rely on offline guess protection from the second PAKE to protect these keys. We formalize a more mild unconditional hiding property, namely that the second PAKE’s transcript does not reveal whether the first PAKE run was successful, that is sufficient to prove the security of **SeqComb**. We identify KEM-based PAKEs with this property, namely CAKE and EKE-PRF [13, 38]. Thus, we get a hybrid PAKE with a round complexity that is the sum of the underlying PAKEs’, and computational overhead of only two hashes. To the authors’ knowledge, this is the first (concurrent with [45]) description of a hybrid PAKE.

Beyond PAKE. Finally, we remark that hybrid PAKE implies hybrid instances of other flavors of PAKE. [27, 47] describe combiners which construct an augmented PAKE (aPAKE) from PAKE, authenticated encryption, and either a signature scheme or KEM, respectively. Similarly, the LATKE identity-binding PAKE (iPAKE) framework [40] takes a generic PAKE and identity-based key exchange (IBKE) protocol. Since hybrid signatures [17], KEMs [11], and IBKEs [40] are known, hybrid PAKE implies hybrid aPAKE and iPAKE.

2 Preliminaries

2.1 Notation

We write probabilistic algorithms as $\text{Alg}(x; r)$ where x denotes the input and r denotes the random coins. We write $y \leftarrow \text{Alg}(x)$ to denote sampling uniform r and setting $y := \text{Alg}(x; r)$, and $x \leftarrow \$ S$ to denote sampling a uniform value from a set S . For our security proofs we will consider probabilistic polynomial time (PPT) adversaries, and denote them with calligraphic letters $\mathcal{A}$. We use λ to denote the security parameter.

In our pseudocode for ideal functionalities, **assert** checks the given condition, and early-returns with “assertion failed” on failure. **retrieve** denotes retrieval of a record with a specific marking; if no such record is present, this early-returns with “no record.” We mark new variables introduced into the scope by a **retrieve** using square brackets. We write $\boxed{\text{P}^{\text{sid}}}$ to denote execution of a PAKE protocol P with session identifier sid .

2.2 Universal Composability

The Universal Composability (UC) model [20] provides an alternative to game-based security definitions. The model frames cryptographic protocols as idealized *functionalities*, which can be thought of as black boxes with a tightly constrained interface to the outside world. In the security game showing Π *UC-realizes* the functionality $\mathcal{F}$, the goal of an interactive Turing machine called the *environment*

$\mathcal{Z}$ is to distinguish between the *ideal world* and the *real world*, which are defined as follows.

Real/Ideal. In the *real world* all the parties participate in Π , $\mathcal{Z}$ may view parties' outputs, and is permitted to give arbitrary instructions to a separate interactive Turing machine, called the *adversary* $\mathcal{A}$. The environment can arbitrarily ask the adversary to view/modify/delay/drop messages between parties, corrupt parties, and interact with any ideal functionalities $\mathcal{F}'_i$ used to instantiate Π . In the *ideal world* all the parties are *dummies*, speaking directly to $\mathcal{F}$. In addition, the adversary may also only interact with $\mathcal{F}$, though it is still permitted to corrupt parties.

Π *UC-realizes* $\mathcal{F}$ if for any $\mathcal{A}$, there exists a simulator $\mathcal{S}$ of $\mathcal{A}$ such that any $\mathcal{Z}$ has at most negligible advantage in distinguishing between $\mathcal{A}$ and $\mathcal{S}$. That is,

$$\text{IDEAL}_{\mathcal{S}, \mathcal{Z}}^{\mathcal{F}} \stackrel{c}{\approx} \text{EXEC}_{\mathcal{A}, \mathcal{Z}}$$

where the LHS refers to the probability ensemble consisting of the view of the environment in the ideal world, and the RHS in the real world.

By [20, Theorem 11], it suffices to imagine that the adversary $\mathcal{A}$ is the *dummy adversary* $\mathcal{A}_D$, which simply delivers backdoor messages generated by the environment to the specified recipients, and delivers to the environment all backdoor messages generated by the protocol parties, as well as the sender machine's identity.

Corruption. There are two models in which the environment can corrupt parties. In the *static corruption model*, the adversary may not corrupt any parties during protocol execution. In the *adaptive corruption model*, the adversary may corrupt parties at any time. In this work, we will consider adversaries who are allowed static corruptions. We note that, since PAKE sessions are so short lived, and static corruptions permit corruptions in between executions, this security model is still quite strong.

Sufficiency of Single-Session Functionality. The ideal functionalities we consider in this work are *single-session*, i.e., they permit at most one key exchange for a single pair of parties, after which they cease to function. It is known that a single-session protocol can be lifted to a multi-session (i.e., multiple pairs of parties, multiple key exchanges per pair) protocol with joint state as long as the multi-session extensions of the helper (formally, *hybrid*) functionalities exist [22, 34]. Since our reductions only use the random oracle functionality, which has a known multi-session extension, we get these theorems for free.

Session Identifiers. To represent different instances of the same scheme, the UC model uses *session identifiers*. Each party is given a session identifier sid on activation and will only interact with other parties with the same sid . For clarity of presentation, we will assume in our protocol definitions that session identifier establishment has already occurred. Further, we will assume that every pair of parties, and hence every execution in the single-session model, has a unique sid (this can be done, e.g., by exchanging random nonces or appending unique personal identifiers to the existing sid).

We note that single-session functionalities do not usually make use of the session identifier sid , since it does not serve to disambiguate sessions. Regardless, we include sid as a random oracle input and ideal cipher domain separator in our pseudocode. We do this so that an implementor who does not read this section will be more likely to implement the protocols securely, i.e., in a way that yields more than single-session security.

2.3 PAKE

A (balanced) password authenticated key-exchange protocol (PAKE) is a two-party key-exchange protocol where parties use mutual knowledge of a low-entropy password pw to establish a high-entropy *session key* K .

2.3.1 PAKE Protocols

We define a rudimentary notion of a PAKE protocol by specifying its input-output behavior and correctness. Looking ahead, the following definition captures the minimal guarantees that we can expect from a single PAKE building block of our combiners, even when its security breaks.

Definition 1 (PAKE protocol). *Let P be an interactive protocol between two parties, $\mathcal{P}$ and $\mathcal{P}'$, producing keys of bit length λ . We say P is a PAKE protocol if it satisfies the following properties.*

1. P takes input pw from $\mathcal{P}$ and pw' from $\mathcal{P}'$
2. P produces output K for $\mathcal{P}$ and K' for $\mathcal{P}'$
3. If $\text{pw} = \text{pw}'$ then $K = K'$ except with probability negligible in λ

For any interactive protocol, a *protocol round* is the set of all messages that can be sent in parallel from any point in the protocol [29]. Thus, in a one-round protocol, no message depends on another. We require a minimal syntax $P = (\text{Start}, \text{Finish})$ for one-round PAKEs, where minimal means that the PAKE may consist of more algorithms (e.g., for computing a setup), and the algorithms may take more inputs. The minimal requirements are as follows:

- **Start** takes as input a security parameter 1^λ , the session identifier sid , and a password pw and outputs a message x and a state st ; and
- **Finish** takes as input st and a message y and outputs a key.

2.3.2 Ideal PAKE Functionality

The security goals for a PAKE are (1) to establish a high-entropy shared session key when both parties are honest, (2) to prevent passive adversaries from learning anything about the password, and (3) to limit active adversaries to one (or another small constant) password guess(es) per protocol instance, even if given access to session keys. These guarantees can be captured by the ideal functionality $\mathcal{F}_{\text{PAKE}}$ [21]. Some widely used protocols use a slight weakening of this

functionality, called *lazy extraction PAKE* ($\mathcal{F}_{\text{lePAKE}}$) [2], which permits adversaries complete active attacks even after a session is already finished. In this work we also make use of a novel functionality, a *relaxed* version of lazy extraction PAKE ($\mathcal{F}_{\text{rlePAKE}}$), where the adversary additionally learns whether the active attack was successful. We state all functionalities in Fig. 1, and include the small modifications from [4] regarding adversary-controlled keys.

Isolated Parties. We introduce another small modification to our PAKE ideal functionality. In the original definition, it is possible for a party to output a session key before the counterparty has even started its session. In our definition, we prevent these *isolated parties* from getting their session key. This modification is crucial to using $\mathcal{F}_{\text{PAKE}}$ in a combiner, since we rely on timely extraction from a particular sub-PAKE, and cannot deal with output keys that were produced without any interaction.

This new functionality is slightly stronger than the original, but this is not a problem: any PAKE which UC-realizes the original $\mathcal{F}_{\text{PAKE}}$ UC-realizes the new one as well. Consider a UC simulator for a such a PAKE. We will show that `NewKey` is never called on an isolated party. First, note if the simulator calls `NewKey` on an isolated party, it is because the environment $\mathcal{Z}$ has triggered the end to the protocol (this is because `NewKey` gives $\mathcal{Z}$ the session key, so $\mathcal{Z}$ knows precisely when `NewKey` is called). Further, if `NewSession` has not been called for the counterparty, it is because $\mathcal{Z}$ has not activated that party. So the only case in which `NewKey` can be called on an isolated party is if only that party was activated and the protocol has finished, i.e., the protocol is at most a one-message PAKE. Since such PAKEs are impossible in the UC model,² this scenario never occurs.

2.3.3 Hiding Properties of PAKEs

For our combiner constructions, we will require two specific security properties from the underlying PAKEs.

Perfect Password Hiding. The first property we require is that of *perfect password hiding*—that PAKE execution transcripts perfectly hide the password (as opposed to relying on a computational indistinguishability assumption). Crucially, we demand this property from PAKE protocols *even in situations where its security breaks*, e.g., through the discovery of an algorithm that solves the underlying computational assumption in polynomial time. Hence, we define a standalone property rather than modifying the ideal PAKE functionality. We

² More specifically, PAKEs with negligible correctness error and at most one message are impossible. An efficient offline attack is as follows. Let $\mathcal{P}$ be the initiator in the execution of a one-message PAKE, sending message m to $\mathcal{P}'$. If $\mathcal{Z}$ permits $\mathcal{P}$ to send m , then $\mathcal{P}'$ immediately outputs its session key K . This K is the output of a function $f(\text{pw}, \text{sid}, m)$ (w.l.o.g., this is deterministic, otherwise we have noticeable correctness error). Since $\mathcal{Z}$ knows sid and m , it can test various pw until it receives an output that matches K .

Session management On (NewSession, sid, $\mathcal{P}_j$, pw_i) from $\mathcal{P}_i$ send (NewSession, sid, $\mathcal{P}_i$, $\mathcal{P}_j$) to $\mathcal{A}$ if $\nexists(\text{Session}, \text{sid}, \mathcal{P}_i, \cdot, \cdot) \wedge \neg \text{NoMoreSessions}$: // After two NewSessions stop accepting more if $\exists(\text{Session}, \dots) : \text{record NoMoreSessions}$ record (Session, sid, $\mathcal{P}_i$, $\mathcal{P}_j$, pw_i) and mark it fresh	Active session attack On (TestPwd, sid, $\mathcal{P}_i$, pw') from $\mathcal{A}$ retrieve (Session, sid, $\mathcal{P}_i$, [$\mathcal{P}_j$], [pw_i]) marked fresh if $\text{pw}_i = \text{pw}'$: Mark session compromised send “correct” to $\mathcal{A}$ else Mark session interrupted send “wrong” to $\mathcal{A}$
Key generation and authentication On (NewKey, sid, $\mathcal{P}_i$, K') from $\mathcal{A}$ retrieve (Session, sid, $\mathcal{P}_i$, [$\mathcal{P}_j$], [pw_i]) with mark $m \neq \text{completed}$ if $m = \text{fresh} : \text{assert } \exists(\text{Session}, \text{sid}, \mathcal{P}_j, \mathcal{P}_i, \cdot)$ if $m = \text{compromised} : K_i := K'$ elif $m = \text{fresh}$ and $\exists(\text{Key}, \text{sid}, [\mathcal{P}_j], [\text{pw}_j], [K_j], \text{fresh})$ s.t. $\text{pw}_i = \text{pw}_j$: $K_i := K_j$ else : $K_i \leftarrow \{0, 1\}^\lambda$ record (Key, sid, $\mathcal{P}_i$, pw_i , K_i , m) Mark the session completed send (sid, K_i) to $\mathcal{P}_i$	Completed session attack On (RegisterTest, sid, $\mathcal{P}$) from $\mathcal{A}$ retrieve (Session, sid, $\mathcal{P}$, $\cdot, \cdot$) marked fresh Mark the record interrupted and flag it tested On (LateTestPwd, sid, $\mathcal{P}$, pw') from $\mathcal{A}$ retrieve (Session, sid, $\mathcal{P}$, $\dots$) flagged tested Remove the flag retrieve (Key, sid, $\mathcal{P}$, [pw], [K], $\cdot$) if $\text{pw} = \text{pw}' : \text{set } K' := K$ else : Set $K' \leftarrow \{0, 1\}^\lambda$ Set $K' := \perp$ send (sid, K') to $\mathcal{A}$

Fig. 1. The $\mathcal{F}_{\text{PAKE}}$, $\mathcal{F}_{\text{lePAKE}}$, and $\mathcal{F}_{\text{rlePAKE}}$ ideal functionalities. The original $\mathcal{F}_{\text{PAKE}}$ functionality excludes all gray and dashed parts. Adding the [dashed] interfaces without the line in dark gray yields the lazy password extraction functionality $\mathcal{F}_{\text{lePAKE}}$. Adding the line in dark gray yields the relaxed lazy extraction functionality $\mathcal{F}_{\text{rlePAKE}}$. In this paper, we include the line in light gray in *all* functionalities, to prevent isolated parties from outputting a key.

restrict the definition to one-round PAKEs for simplicity and because the popular PAKEs satisfying this property are all one-round anyway.³

We formally define perfect password hiding through the existence of a simulator that produces identically distributed protocol messages without access to the password, but is still able to compute keys from simulated messages and passwords, using a simulation trapdoor. We note that the existence of a simulator for producing *computationally* indistinguishable transcripts follows from the fact that a PAKE protocol UC-realizes $\mathcal{F}_{\text{PAKE}}$ or $\mathcal{F}_{\text{lePAKE}}$. Essentially, perfect password hiding demands that this part of the PAKE never breaks.

Definition 2 (Perfectly password-hiding one-round PAKE). *Let $P = (\text{Start}, \text{Finish})$ be a one-round PAKE protocol. We call P perfectly password hiding iff there exist PPT algorithms $\text{Sim} = (\text{Start}, \text{ComputeKey})$ such that:*

³ In fact, we do not know of a PAKE with this property that is not one round. The failure modes were discussed briefly in the introduction, using KC-SPAKE2 and CAKE as examples.

- **Sim.Start** is a probabilistic algorithm that takes as input a security parameter 1^λ and session identifier sid , and outputs a tuple (x, τ)
- **Sim.ComputeKey** is a deterministic algorithm that takes as input $(\text{sid}, x, y, \tau_y, \text{pw})$ and outputs a key
- **Perfect hiding.** For any pw , $\text{P.Start}(1^\lambda, \text{sid}, \text{pw})|_1$ is identically distributed, where $|_1$ denotes the first component of the output. In other words, every message in the honest protocol is equally explained by any password.
- **Trapdoor key computation.** For any pw , $(x, \text{st}) := \text{P.Start}(1^\lambda, \text{sid}, \text{pw})$, and $(y, \tau_y) := \text{Sim.Start}(1^\lambda, \text{sid})$, it holds that $\text{Sim.ComputeKey}(x, y, \tau_y, \text{pw}) = \text{P.Finish}(\text{st}, y)$.

For brevity, we will avoid writing sid in the inputs of **P.Start** and **Sim.Start** when it is otherwise clear.

EKE, CPace, and SPAKE2 (Fig. 2) are perfectly hiding one-round PAKEs. This property follows immediately from the fact they have UC proofs and their messages (Diffie-Hellman key shares) are uniformly distributed for any fixed password, i.e., any password equally explains any message. To demonstrate, we provide the proof for CPace below.

CPace is Perfectly Password-Hiding. We will assume the group is prime-order for simplicity, though CPace is also secure for groups with nontrivial cofactor. We define **Sim** as follows. First, we let the simulator respond to H_0 queries by selecting a scalar u , recording u , and returning uB where B is a fixed generator. **Sim.Start** picks a uniform scalar r and computes $R := rB$. The output is $(\text{sid}, x = R, \tau = r)$.

Sim.ComputeKey takes r, R, S, pw . Let u' be the recorded discrete log of $H_0(\text{sid}, \text{pw})$. Let $r' := r/u'$ and let $Z := r'S$. **Sim.ComputeKey** outputs $H_1(\text{sid}, R, S, Z)$.

This satisfies the trapdoor key computation requirement: if $S = s \cdot H_0(\text{sid}, \text{pw})$ is honestly generated, then

$$Z = r'S = r's \cdot H_0(\text{sid}, \text{pw}) = r'su'B = rsB = sR.$$

This is also perfectly hiding, since **P.Start** outputs a uniformly random curve point.

PSK Equality Hiding. The second property we require is that of *statistical PSK equality hiding*—that a PAKE, when executed by two parties using high-entropy passwords (a.k.a., a preshared key, or PSK), does not leak whether the passwords match. As before, we require this property even when the PAKE is otherwise insecure. Note that this is a strictly weaker property than perfect password hiding. We provide a proper definition below. Unlike with password hiding, we define this property to include multi-round PAKEs.

Definition 3 (PSK Equality Hiding). *Let P be a PAKE protocol. We say that P is PSK equality hiding iff an active adversary observing two honest parties interacting cannot distinguish between the case where the parties share a high-entropy password and the case where they have distinct high-entropy passwords.*

<u>Alice(sid, pw)</u>		<u>Bob(sid, pw)</u>
$G := H_0(\text{sid}, \text{pw})$		$G := H_0(\text{sid}, \text{pw})$
$r \leftarrow \$ \mathbb{F}$		$s \leftarrow \$ \mathbb{F}$
$R := rG$	$\xrightleftharpoons[S]{R}$	$S := sG$
$Z := rS$		$Z := sR$
$K := H_1(\text{sid}, R, S, Z)$		$K := H_1(\text{sid}, R, S, Z)$
return K		return K

(a) The CSpace PAKE [32]. The random oracle H_0 outputs values in a prime-order group.

<u>Alice(sid, pw)</u>		<u>Bob(sid, pw)</u>
$r \leftarrow \$ \mathbb{F}$		$s \leftarrow \$ \mathbb{F}$
$R := E_{\text{pw}}^{\text{sid}}(rG)$	$\xrightleftharpoons[S]{R}$	$S := E_{\text{pw}}^{\text{sid}}(sG)$
$Z := rD_{\text{pw}}^{\text{sid}}(S)$		$Z := sD_{\text{pw}}^{\text{sid}}(R)$
$K := H_1(\text{sid}, R, S, Z)$		$K := H_1(\text{sid}, R, S, Z)$
return K		return K

(b) The EKE PAKE [14,38]. G is a group generator, and (E, D) are the encryption and decryption algorithms of an ideal cipher.

<u>Alice(sid, pw)</u>		<u>Bob(sid, pw)</u>
$r \leftarrow \$ \mathbb{F}$		$s \leftarrow \$ \mathbb{F}$
$R := rG + \text{pw} \cdot M$	$\xrightleftharpoons[S]{R}$	$S := sG + \text{pw} \cdot N$
$Z := r \cdot (S - \text{pw} \cdot N)$		$Z := s \cdot (R - \text{pw} \cdot M)$
$K := H_1(\text{sid}, \text{pw}, R, S, Z)$		$K := H_1(\text{sid}, \text{pw}, R, S, Z)$
return K		return K

(c) The SPAKE2 PAKE [6]. G , M , and N are unrelated generators of a prime-order group. pw is interpreted as an integer (perhaps via hashing).**Fig. 2.** Three perfectly password-hiding PAKEs.

Formally, we define the advantage of an adversary $\mathcal{A}$ as

$$\text{Adv}_{\text{P}}^{\text{PEH}}(\mathcal{A}) = |2 \cdot \Pr[\text{G}_{b,\text{P}}^{\text{PEH}}(\mathcal{A}) = b \mid b \leftarrow \$ \{0, 1\}] - 1|,$$

where G^{PEH} is the PSK equality hiding game defined in Fig. 3. Note we permit the adversary to read and modify messages, but not see the final protocol output.

We say that P is statistically PSK equality hiding if $\text{Adv}_P^{\text{PEH}}(\mathcal{A}) = \text{negl}(\lambda)$ for any unbounded $\mathcal{A}$.

Game $G_{b,P}^{\text{PEH}}(\mathcal{A})$ $\text{psk}_0 \leftarrow \{0, 1\}^\lambda$ if $b = 0$: $\text{psk}_1 \leftarrow \{0, 1\}^\lambda$ else : $\text{psk}_1 := \text{psk}_0$ $i := 0$ // wlog the initiator is participant 0 $(\text{st}_0, \text{st}_1) := (P_0.\text{Start}(\text{psk}_0), P_1.\text{Start}(\text{psk}_1))$ $\text{st}_{\mathcal{A}} := \emptyset$ $m := \emptyset$	while $(\text{done}_0, \text{done}_1) \neq (\text{true}, \text{true})$: $(\text{st}'_i, \text{done}'_i, m') := P_i.\text{Next}(\text{st}_i, m)$ $(\text{st}'_{\mathcal{A}}, m) := \mathcal{A}(\text{st}_{\mathcal{A}}, m')$ $\text{st}_i := \text{st}'_i$ $\text{st}_{\mathcal{A}} := \text{st}'_{\mathcal{A}}$ $\text{done}_i := \text{done}'_i$ $i := i \oplus 1$ $b' := \mathcal{A}(\text{st}_{\mathcal{A}}, \text{final})$ return $b = b'$
---	---

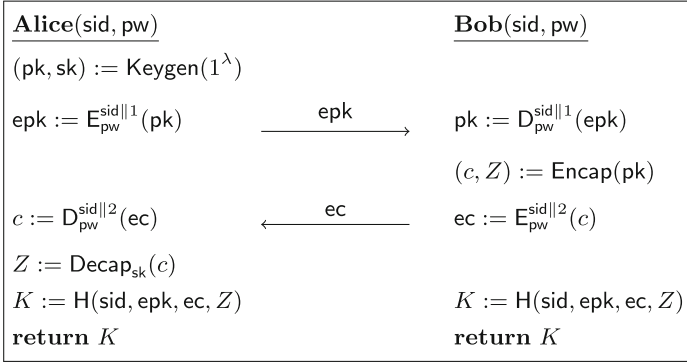
Fig. 3. The PSK equality hiding (PEH) security game for the PAKE P . $\mathcal{A}$ may read and modify any messages, and has access to any ideal functionality, e.g., a random oracle, that P has access to.

As with password hiding, the computational version of PSK equality hiding is guaranteed by the fact that the PAKE UC-realizes one of the PAKE functionalities, but the statistical version is not.

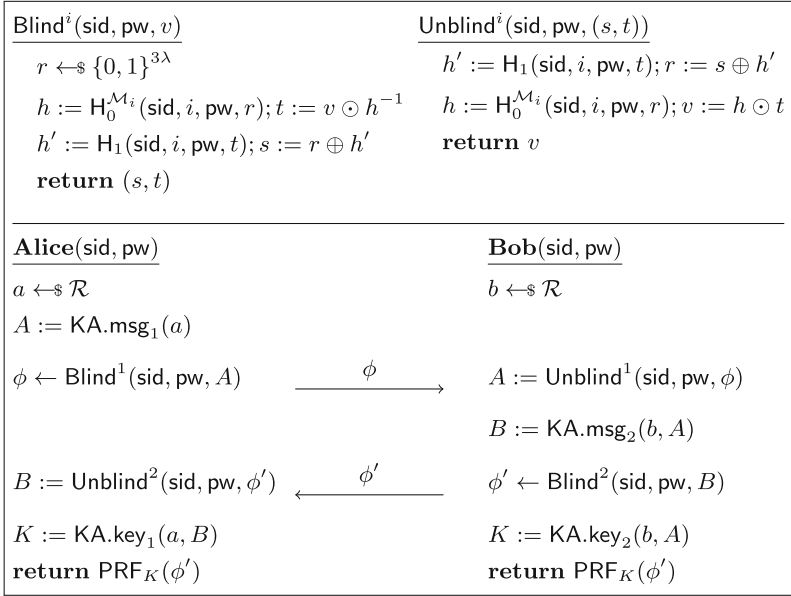
Fortunately, some recent efficient universally composable PAKEs such as CAKE [13] and EKE-PRF [38] (Fig. 4) achieve this property. When instantiated with an appropriate post-quantum KEM, and post-quantum key agreement protocol, respectively, these constructions yield post-quantum PAKEs. At a high level, this property holds because both protocols are two rounds, and in each round, both messages are encrypted under the password using independent ideal ciphers or random oracles. In other words, the parties in these protocols are communicating over a confidential channel.

We provide the formal PSK equality hiding argument for CAKE below. We note that proving this for OEKE-style PAKEs such as OEKE-PRF, OCAKE, CHIC, and NoIC [9, 10, 13, 38], is more subtle and would require a careful analysis of whether the unencrypted second message reveals statistical differences in uniform versus honestly generated KEM public keys.⁴

⁴ There are KEMs for which these constructions are provably *not* statistically PSK-equality-hiding. Consider Regev’s public key encryption scheme [51] as an IND-CPA-secure KEM. In all the aforementioned protocols, an encapsulated key is sent in the clear. Consider the case where the (high-entropy) passwords are equal, i.e., encapsulation is to an honestly generated public key. In Regev’s scheme, every encrypted 0 bit is an LWE sample, and every encrypted 1 bit is not. So an adversary with an LWE oracle will observe 50% LWE samples. However, in the case where the passwords are unequal, encapsulation is to a random element in the space of public keys. With overwhelming probability *none* of the encrypted bits are LWE samples. Thus, an adversary can win the PEH game if given an LWE oracle.



(a) The CAKE KEM-based PAKE [13]. (E, D) are the encryption and decryption algorithms of an ideal cipher.



(b) The EKE-PRF PAKE [38]. KA is a two-round key agreement protocol with randomness space $\mathcal{R}$, message spaces $\mathcal{M}_1, \mathcal{M}_2$ (first and second messages, respectively), both of which are abelian groups (e.g., the module lattice used in ML-KEM, with addition) with group operation denoted $\odot$. The random oracle $H_0^{\mathcal{M}_i}$ takes on values in $\mathcal{M}_i$ and H_1 takes on values in $\{0, 1\}^{3\lambda}$. PRF is a pseudorandom function.

Fig. 4. Two post-quantum statistically PSK-equality-hiding PAKEs.

CAKE is PSK-Equality-Hiding. Let game 0 be the PEH game with $b = 0$, i.e., the passwords are different. In game 1, rather than sending $ec := E_{pw_A}^{sid||1}(pk)$, Bob sends a uniformly chosen ciphertext. Since the ideal cipher is a family of uniformly chosen permutations, and both ideal cipher instances are independent, the advantage of this hop is bounded by the ability of the adversary to guess pw_B , which is $2^{-\lambda}$ times the number of ideal cipher queries.

In game 2, replace Bob's copy of pk with a uniformly chosen public key. Since the ciphertext c is discarded anyway, this hop is perfect.

In game 3, replace Alice's copy of epk with a uniformly random public key. This does not affect Bob's computations, since epk is discarded. Further, since the ideal cipher is a family of uniformly chosen permutations and the instances are independent the advantage of this hop is bounded by the ability of the adversary to guess pw_A , which is $2^{-\lambda}$ times the number of ideal cipher queries.

In game 4, we set $b = 1$, i.e., we make Alice and Bob's passwords equal. This does not affect the transcript or any computations.

In games 5, 6, and 7, we undo games 3, 2, and 1, respectively, and we are done.

EKE-PRF is PSK-Equality-Hiding. This argument operates nearly identically. Since EKE-PRF includes many more intermediate variables, we present a sketch.

We may replace ϕ' with randomness, since both components are perfectly blinded with independent outputs of a random oracle (namely, h and h'), whose input includes pw_B , which is high-entropy. We may then replace Bob's copy of A with randomness, since no function of Bob's A appears in the transcript anymore. We then replace ϕ with randomness, since it does not affect Bob's computation, and both components are perfectly blinded with independent outputs of a random oracle whose input includes pw_A , which is high-entropy. We then set the passwords equal, which does not affect anything, and then we undo all the above steps in reverse order.

3 Constructions

In this section we present our two PAKE combiners, **ParComb** (*parallel combiner*) and **SeqComb** (*sequential combiner*). Unlike the failed attempt at a combiner in our introduction, these combiners will require specific hiding properties of their underlying PAKEs. With these mild assumptions, we show that both combiners achieve security in the UC model.

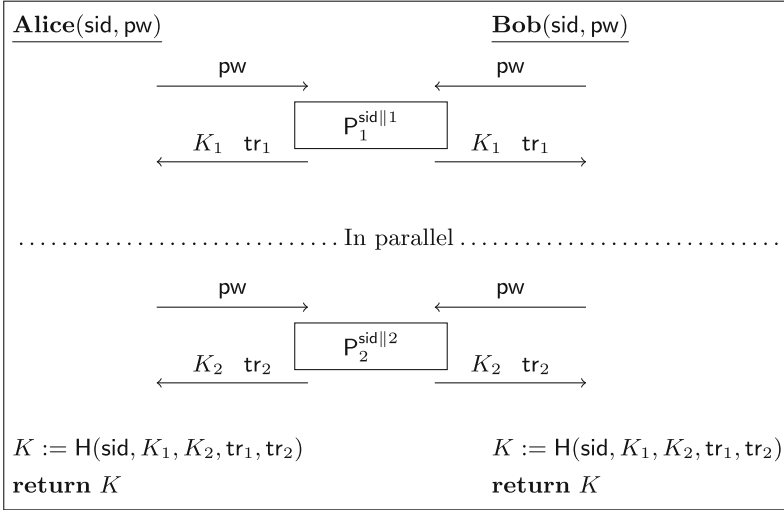


Fig. 5. The ParComb combiner for perfectly password-hiding one-round PAKEs P_1 and P_2 . tr_i is the protocol transcript of the execution of P_i . The random oracle H outputs values in $\{0, 1\}^\lambda$. sid is unique to every execution.

3.1 The ParComb Combiner

We return to the insecure parallel combiner of our introduction. Recall the reason it fails is, if the underlying security assumption were broken, the transcripts of the underlying PAKEs would contain enough auxiliary information to check password guesses. For KC-SPAKE2, this auxiliary information is a MAC whose key is derived from the Diffie-Hellman shared secret, and for CAKE, this is the fact that KEM ciphertexts are LWE samples.

However, not every PAKE has this auxiliary information. In fact, some widely deployed, well-standardized PAKEs such as SPAKE2 or CPace [5, 42] do not. The lack of such auxiliary information is what we formalize in Definition 2 (perfect password hiding). As it turns out, this property is necessary and sufficient in order to make our parallel combiner work. At a high level, ParComb executes two perfectly password-hiding one-round PAKEs P_1 and P_2 in parallel, and hashes the resulting keys. This combiner is highly efficient, requiring only one extra hash, and has round complexity bounded by the maximum round complexity of the underlying PAKEs. We give the full construction in Fig. 5.

3.1.1 Security

Before presenting the security theorem, we provide intuition for each assumption and design decision.

Perfect Password Hiding. We know from our broken combiner example why perfect (or at least statistical) password hiding is necessary for P_1 and P_2 . We

provide intuition now for why it is sufficient. Consider what happens when P_1 is broken and P_2 remains secure. An active adversary can engage in P_1 with Bob and be able to efficiently compute the guessing function $\text{pw} \mapsto K'_1$ (in EKE, for example, this requires solving computational Diffie-Hellman). However, since the messages are perfectly password-hiding, the only way to check a guess is to compare it to the output key $K := H(\text{sid}, \text{tr}, K_1, K_2)$. So the adversary is in the scenario where it can make guesses of pw and see (a hash of) K_2 . This is precisely the UC PAKE game for P_2 , which is assumed to be secure.

The same argument shows that P is secure when P_2 is broken and P_1 remains secure.

Hashing the Transcript. We include the full protocol transcript in the final key computation so that the session key can be simulated when messages are mauled. Suppose P_1 is broken, and the adversary has modified one of the P_1 messages. If a simulator does not know either party's password, then it cannot decide whether P_1 should succeed or fail. And since P_1 is broken, the simulator cannot rely on the PAKE UC-realizing $\mathcal{F}_{\text{PAKE}}$. Hashing the transcript resolves the ambiguity, since any mauled messages will cause the protocol to fail (i.e., return different keys) with overwhelming probability.

We now state the security theorem for **ParComb**. The arguments of the proof are sketched above, and the proof can be found in the full version of this paper [35].

Theorem 1. *Let P_1 and P_2 be perfectly password-hiding one-round PAKE protocols, and let $P := \text{ParComb}[P_1, P_2]$. Then the following hold in the static-corruptions setting:*

1. *If P_1 UC-realizes $\mathcal{F}_{\text{PAKE}}$, then so does P (in the $\mathcal{F}_{\text{RO}}$ -hybrid model)*
2. *If P_2 UC-realizes $\mathcal{F}_{\text{PAKE}}$, then so does P (in the $\mathcal{F}_{\text{RO}}$ -hybrid model).*

In other words, $\text{ParComb}[P_1, P_2]$ is at least as strong as the strongest of P_1, P_2 , and is hence a good combiner.

3.1.2 Instantiations

(Not) Achieving $\mathcal{F}_{\text{PAKE}}$. While this combiner's efficiency and round complexity are appealing, applicability is currently limited. One of the underlying PAKEs can be straightforwardly instantiated with EKE [14], since it is one-round, perfectly password hiding, and UC-realizes $\mathcal{F}_{\text{PAKE}}$ [38]. The other underlying PAKE remains a question, though, as there is no known post-quantum PAKE with these properties.

One line of work includes the EKE-NIKE construction from [12], using the lattice-based NIKE, Swoosh [26]. However, this does not have perfect hiding, since messages are of the form $E_{\text{pw}}(\mathbf{t})$, where $\mathbf{t}$ is an LWE sample and E is an ideal cipher. This has the same problem as the CAKE construction in the introduction.⁵ Thus, we do not yet have a one-round hybrid construction realizing $\mathcal{F}_{\text{PAKE}}$.

⁵ While it appears no post-quantum PAKE has this property, we do not believe it is inherent. See Sect. 4 for potential new directions.

Achieving $\mathcal{F}_{\text{blePAKE}}$. A concurrent work [45] shows that **ParComb** can be a *bare lazy-extraction PAKE* (blePAKE), i.e., a lazy-extraction PAKE that does not need a unique sid . Specifically, the authors show that, when **ParComb**'s inputs consist of lePAKEs and at least one blePAKE (both with statistical password hiding), then the combination UC-realizes $\mathcal{F}_{\text{blePAKE}}$. They also show that X-GA-PAKE (isogeny-based), SPAKE2 (ECDH-based), and TBPEKE (ECDH-based) [3, 6, 50], are blePAKEs with statistical password hiding. Thus, **ParComb** can be instantiated as a one-round hybrid construction realizing $\mathcal{F}_{\text{blePAKE}}$.

3.2 The SeqComb Combiner

To arrive at the **SeqComb** combiner, we again consider the minimum possible amount of security necessary for a combined PAKE. Beginning our protocol with a one-round perfectly password hiding PAKE cannot hurt, since even when it is broken, it is difficult to exploit for a password guess. To handle the case that this PAKE P_1 is broken, we may pass pw along with the P_1 's session key to a second PAKE P_2 , receive a second session key, and output the hash of everything. If P_2 is secure, then including its session key in the final hash will be sufficient to ensure that the overall protocol is secure. If the P_2 is broken then including P_1 's session key in the final hash will be sufficient, so long as P_2 is not so broken as to reveal pw outright.

This is almost exactly the definition of **SeqComb**. That is, **SeqComb** executes a perfectly password-hiding PAKE P_1 , feeds its output into a statistically PSK equality hiding PAKE P_2 , and hashes everything at the end. Like **ParComb**, this combiner is highly efficient, requiring only two extra hashes. Unlike **ParComb**, its round complexity is the sum of the number of rounds of the underlying PAKEs. We give the full construction in Fig. 6.

3.2.1 Security

Again, before presenting the security theorem, we provide intuition for why each component of the protocol is necessary.

P_1 Perfect Password Hiding. We require P_1 to be perfectly password hiding for the same reason as in **ParComb**: it directly deals with pw , and so any leakage here yields a leakage in the combined protocol. More rigorously, if P_1 is broken, an active adversary can efficiently compute the mapping $\text{pw} \mapsto Z'$, but not receive any extra information on whether its guess was correct. So the adversary is in the scenario where it has guesses for Z and can see (a hash of) K_2 . This is precisely the UC PAKE game for P_2 .

P_2 Statistical PSK Equality Hiding. We require P_2 to be statistically PSK equality-hiding in order to permit its transcript to be simulated when P_2 is broken. In the case where both **SeqComb** parties are honest, and have completed P_1 , a simulator must simulate P_2 without knowing whether P_1 succeeded, i.e., without knowing whether the parties agree on the input to P_2 . Since this input is high-entropy by assumption (P_1 is a secure PAKE), then the PSK equality

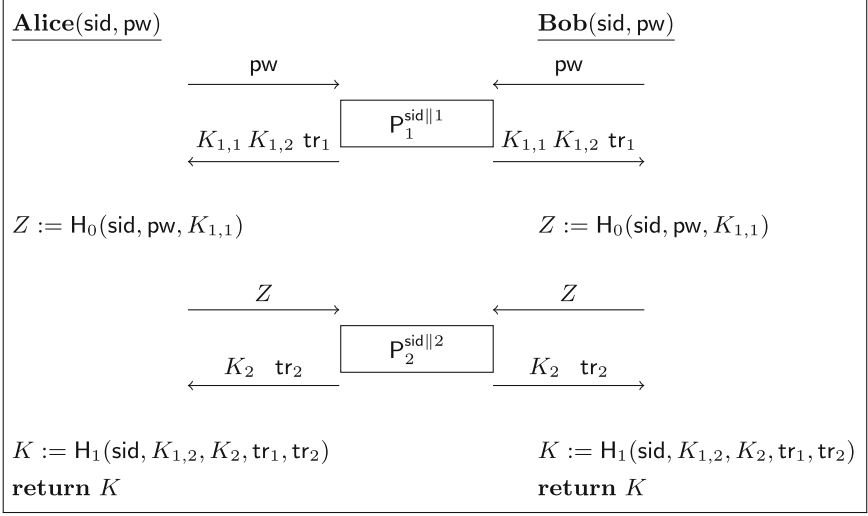


Fig. 6. The SeqComb combiner for PAKEs P_1, P_2 , where P_1 is one round and perfectly password-hiding, and P_2 is statistically PSK equality-hiding. tr_i is the protocol transcript of the execution of P_i . Both random oracles H_0, H_1 output values in $\{0, 1\}^\lambda$. sid is unique to every execution. We assume P_1 outputs two keys, each of bitlength λ (if not, it suffices to apply a PRG to stretch its output).

hiding property is sufficient to permit the simulator to simulate one case and have it be indistinguishable from the other.

Hashing $K_{1,1}$. We include $K_{1,1}$ in the hash Z because pw alone is insufficient in the case that P_2 is broken. If P_2 were so broken as to leak its password Z in its entirety (technically prevented by PSK equality hiding, but we may assume), then pw would be trivially revealed. We therefore hash $K_{1,1}$, which we assume is high entropy, into Z . Thus, even if Z leaks in its entirety, pw is unaffected.

Hashing pw . We include pw in the computation of Z because $K_{1,1}$ is not sufficient in the case that P_1 is broken. If P_1 is broken, then we must pessimistically assume that both $K_{1,1}$ and $K_{1,2}$ are fully controlled by the adversary. Thus, simply feeding one of these values in P_2 yields no security. Instead, we hash pw with $K_{1,1}$ and feed that into P_2 .

Hashing $K_{1,2}$. We include $K_{1,2}$ in the final session key computation because K_2 is not sufficient in the case that P_2 is broken. It suffices to include some yet-unused entropy from the session key of P_1 .

We now state the security theorem for SeqComb. The arguments of the proof are sketched above, and the proof can be found in the full version of this paper [35].

Theorem 2. *Let P_1 be a one-round perfectly password-hiding PAKE protocol, let P_2 be a statistically PSK equality hiding PAKE protocol, and let $P := \text{SeqComb}[P_1, P_2]$. Then the following hold in the static-corruptions setting:*

1. *If P_1 UC-realizes $\mathcal{F}_{\text{PAKE}}$, then so does P (in the $\mathcal{F}_{\text{RO}}$ -hybrid model).*
2. *If P_2 UC-realizes $\mathcal{F}_{\text{PAKE}}$, then so does P (in the $\mathcal{F}_{\text{RO}}$ -hybrid model).*

In other words, $\text{SeqComb}[P_1, P_2]$ is at least as strong as the strongest of P_1, P_2 , and is hence a good combiner.

3.2.2 Using Weaker P_1

Some of the most widely deployed PAKEs such as the Diffie-Hellman-based CPace and SPAKE2 do not enjoy full UC PAKE security but are so-called *lazy extraction* PAKEs. These PAKEs allow adversaries to successfully complete an active attack even after the attacked party finished the protocol. Such prolonged attack completions do not seem to pose any real-world threat, and lazy extraction PAKEs are in widespread use (e.g., for Facebook Messenger chat history transfer [24]). Hence, to make SeqComb more applicable, we would like to make it work when P_1 UC-realizes $\mathcal{F}_{\text{lePAKE}}$. This functionality is depicted in Fig. 1 and is strictly weaker than $\mathcal{F}_{\text{PAKE}}$ due to the additional password test interface on completed sessions. This interface allows the “lazy” attacker to provide a password guess even after an actively attacked party output a session key, and it provides the attacker with that key or a random one depending on whether the guess was correct.

Perhaps surprisingly, SeqComb is not as tolerant of a lazy-extraction P_1 as one would hope for: $\text{SeqComb}[\mathcal{F}_{\text{lePAKE}}, P_2]$ does not UC-realize $\mathcal{F}_{\text{lePAKE}}$. This can be made more intuitive by looking at SeqComb ’s design. In the case where P_2 cannot be trusted to protect K_2 , security is based solely on the output keys $K_{1,1}, K_{1,2}$ of P_1 . When these values are revealed to the environment through a correct late password guess on $\mathcal{F}_{\text{lePAKE}}$, the simulator has to explain how they lead to the actual Z and output key K of the honest party. On the other hand, if the guess was wrong, no connection should exist between $K_{1,1}, K_{1,2}$ and Z, K . Since $\mathcal{F}_{\text{lePAKE}}$ does not leak the information of whether the guess was correct or not, the simulator is trapped. We formalize this intuition in Appendix A with a formal distinguisher between SeqComb with a lazy extraction P_1 and $\mathcal{F}_{\text{lePAKE}}$.

This intuition implies a fix. It would suffice to let the simulator know whether a late password guess succeeded. A PAKE that allows one late password guess with responses of the form “correct” or “wrong” is called a *relaxed* PAKE [2]. Combining this property with the lazy extraction property yields what we call *relaxed lazy extraction PAKE*, or rlePAKE . We depict the functionality in Fig. 1. It has the same guarantees as $\mathcal{F}_{\text{lePAKE}}$, but additionally leaks the information whether or not the late password guess was correct—a weakening that does not seem to rule out any applications of PAKE protocols. We prove the following result in the full version of this paper [35].

Lemma 1. *Let P_2 be a statistically PSK equality hiding PAKE protocol, and let $P := \text{SeqComb}[P_1, P_2]$. If P_1 UC-realizes $\mathcal{F}_{\text{lePAKE}}$, then P UC-realizes $\mathcal{F}_{\text{rlePAKE}}$ (in the $\mathcal{F}_{\text{RO}}$ -hybrid model) in the static-corruptions setting.*

Lemma 1 implies that SeqComb is safe to use with a lazy extraction PAKE such as CPace or SPAKE2 as P_1 .

3.2.3 Concrete Instantiations Yielding Hybrid PAKE

SeqComb can be instantiated with existing efficient protocols to produce a hybrid (rle)PAKE. For P_1 , it is possible to use the Diffie-Hellman-based CPace, SPAKE2, or EKE. For P_2 , any statistically PSK equality-hiding PAKE will do, e.g., any plausibly post-quantum EKE-style protocols such as CAKE and EKE-PRF. We discuss the open problem of proving statistical PSK equality hiding for more PAKEs in Sect. 4.

3.2.4 Achieving Security in the QROM

We note that our combiners require random oracles to function, and our instantiations include post-quantum PAKEs that rely on the ideal-cipher and/or random oracle model. Since our goal is (hybrid) post-quantum security, it is important to consider models which more accurately depict the capabilities of a quantum adversary.

SeqComb was analyzed in the Quantum Random Oracle Model (QROM) in concurrent work [45]. The authors show that when P_1 (a perfectly password-hiding PAKE) and P_2 UC-realize $\mathcal{F}_{\text{PAKE}}$, SeqComb *almost UC-realizes* $\mathcal{F}_{\text{PAKE}}$ in the $\mathcal{F}_{\text{QRO}}$ -hybrid model. “Almost” indicates that the simulation proof is made with respect to a restricted set of environments, namely those that never start the protocol with mismatching passwords. While this does not yield full UC emulation and hence does not allow one to apply, e.g., the UC composition theorem to their construction, it does relax the requirements on the underlying PAKE. Namely, P_2 can be an *arbitrary* PAKE, not just one with statistical PSK equality hiding. As a consequence, their (standalone) SeqComb can be instantiated with a classical PAKE like EKE, and a post-quantum PAKE proven in the QROM, such as $\text{PAKE}_{\text{lwe}}^{\text{QRO}}$ [46]. In that sense, our work shows that the gap between standalone and universally composable security of SeqComb is exactly PSK equality hiding of the second PAKE.

For ease of reference, we provide a breakdown of all our results along with those of concurrent work in Table 1.

4 Future Work

We identify some interesting problems which we do not address in this work.

Adaptive Security. So far, we have only defined hybrid PAKE in the static corruptions setting. This may be inherent to the combiner construction. No one-round PAKE can achieve security in the UC model with adaptive corruptions.

Table 1. A summary of combiner results from this work and concurrent work [45]. The +PH marker indicates the PAKE must be perfectly password hiding. The +PEH marker indicates the PAKE must be statistically PSK equality hiding. ($\mathcal{F}$) indicates the analysis was done w.r.t. a restricted set of UC environments.

Combiner	P_1	P_2	Combined Model	
ParComb	$\mathcal{F}_{\text{PAKE}}$	$\mathcal{F}_{\text{PAKE}}$	$\mathcal{F}_{\text{PAKE}}$	ROM Theorem 1
	$\mathcal{F}_{\text{blePAKE}}$	$\mathcal{F}_{\text{lePAKE}}$	$\mathcal{F}_{\text{blePAKE}}$	ROM [45, Thm. 2]
SeqComb	$\mathcal{F}_{\text{PAKE}} + \text{PH}$	$\mathcal{F}_{\text{PAKE}} + \text{PEH}$	$\mathcal{F}_{\text{PAKE}}$	ROM Theorem 2
	$\mathcal{F}_{\text{lePAKE}} + \text{PH}$	$\mathcal{F}_{\text{PAKE}} + \text{PEH}$	$\mathcal{F}_{\text{lePAKE}}$	ROM Lemma 1
	$\mathcal{F}_{\text{PAKE}} + \text{PH}$	$\mathcal{F}_{\text{PAKE}}$	$(\mathcal{F}_{\text{PAKE}})$	QROM [45, Thm. 4]

A simple attack is as follows. The environment $\mathcal{Z}$ initiates a session between an honest Alice and Bob. $\mathcal{Z}$ forwards Alice’s message to Bob, allowing Bob to finish the protocol and output K , and withholds Bob’s message to Alice. Now, $\mathcal{Z}$ corrupts Alice. There is no way for a simulator to respond to this corruption. This is because the simulator must have used $\mathcal{F}_{\text{PAKE}}.\text{NewKey}$ to key Bob, since both parties were honest at the time. Thus, the simulator does not know K , and so cannot choose an appropriate response to $\mathcal{Z}$ ’s corruption query. We conclude that any PAKE combiner which permits one-round PAKEs is limited to static corruptions.

We thus ask whether there exists a method for constructing adaptively secure PAKEs and, if so, whether it can be done generically.

PSK Equality Hiding for more PAKEs. We show statistical PSK equality hiding for two EKE-style PAKEs, i.e., two-round PAKEs wherein both messages are encrypted. We do not know, however, how to show this property for OEKE-style PAKEs, i.e., two-round PAKEs with only one encrypted message [8–10, 19, 38, 48, 49, 53]. Since this is a large class of protocols, it would be useful to establish the properties necessary of the underlying cryptographic primitive (e.g., the structure of an MLWE-based KEM) to achieve statistical hiding.

One-Round Hybrid PAKE. In our assessment of ParComb, we concluded that the combiner cannot achieve $\mathcal{F}_{\text{PAKE}}$ using a post-quantum PAKE, since none has perfect password hiding, to the authors’ knowledge. This does not seem inherent, though. We conjecture that it is possible to modify EKE-NIKE[Swoosh] [12, 26] to perfectly hide the password. This would involve sending two pairs of public keys, replacing public keys of the form $\mathbf{As} + \mathbf{e}$ with ones of the form $\mathbf{As}$ for $\mathbf{A}, \mathbf{s}, \mathbf{e}$ of appropriate dimension. Effectively, this undoes the size improvements of [44] in exchange for statistically uniform public keys. Further, since the uniformity result relies on the leftover hash lemma [33], which is not known to apply in the cyclotomic rings of typical module lattice constructions, the NIKE must use unstructured LWE. We estimate public key sizes in the resulting scheme to be 1GB at the 128-bit security level.

We leave as an open question whether such a scheme could be instantiated and improved on to be more practical.

Analysis Against Quantum Adversaries. While our combiners can, in principle, use PAKEs that are secure against quantum adversaries, our reductions are classical and exist in the ROM. As mentioned earlier, concurrent work [45] analyzes SeqComb in the QROM and proves a theorem with respect to a restricted set of environments. We believe that this difference is small enough that the analysis may transfer immediately to our unrestricted environment setting, yielding a proof in the $\mathcal{F}_{\text{QRO}}$ -hybrid model. We leave this for future work.

Combiners for *Strong* PAKE Variants. As mentioned earlier, hybrid PAKE implies hybrid aPAKE and iPAKE via the Ω -method [27, 47] and LATKE [40]. However, it is not clear how to achieve the *strong* variants of these constructions (called *saPAKE* and *siPAKE*, respectively). A strong aPAKE is an aPAKE which is resistant to precomputation attacks—an adversary must compromise a server before they are able to begin brute-forcing the password. Existing constructions such as OPAQUE [39] use an oblivious pseudorandom function (OPRF) to achieve strongness. However, no hybrid OPRF construction is currently known, and so it is not clear how to hybridize OPAQUE. More generally, it would be interesting to understand how much strongness relies on an OPRF-like construction, and how much can be achieved using generic combiners. For (s)aPAKE and (s)iPAKE in general, such black-box combiners would further be desirable in order to give developers the option to plug post-quantum protocols into their existing deployments.

Acknowledgements. We thank Phillip Gajland for the suggestion of the information-theoretic variant of Swoosh. We thank You Lyu and Shengli Liu for their helpful feedback after we found out we all wrote more or less the same paper. We also thank Jelle Vos and Stanislaw Jarecki (both supported by Apple) for catching an error in our proofs and pointing us to subtleties in proving PSK equality hiding for OEKE-style post-quantum PAKEs. Julia wants to say: thank you, Michael, for sharing your inspiring ideas with me—this has been an awesome project!

A Formal Distinguisher for SeqComb with Lazy Extraction

To make SeqComb work with lazy extraction PAKEs, we would need to prove the following statement: if P_1 UC-realizes $\mathcal{F}_{\text{lePAKE}}$ and P_2 is PSK equality hiding, then P UC-realizes $\mathcal{F}_{\text{lePAKE}}$ (in the $\mathcal{F}_{\text{RO}}$ -hybrid model). In this section we give a formal distinguisher for this statement.

The PSK equality hiding property does not rule out offline attacks against P_2 . We use that to distinguish the protocol from $\mathcal{F}_{\text{lePAKE}}$.

1. $\mathcal{Z}$ starts honest $\mathcal{P}$ with pw , sends RegisterTest to $\mathcal{F}_{\text{lePAKE}}^{(1)}$ and completes $\mathcal{P}$'s session to receive output key K .

2. $\mathcal{Z}$ flips a coin b and sends pw_b as `LateTestPwd` guess to $\mathcal{F}_{\text{lePAKE}}^{(1)}$, where $\text{pw}_0 = \text{pw}$ and pw_1 is a different password.
3. $\mathcal{Z}$ receives key $K_{1,1}, K_{1,2}$ from $\mathcal{F}_{\text{lePAKE}}^{(1)}$
4. $\mathcal{Z}$ tests whether $H_0(\text{sid}, \text{pw}_b, K_{1,1})$ was used by $\mathcal{P}$ in P_2 .
 - If $b = 0$ $\mathcal{Z}$ expects that test to pass
 - If $b = 1$ $\mathcal{Z}$ expects that test to fail.

A simulator Sim can react in the following way to this $\mathcal{Z}$

1. Sim has no information about $\mathcal{P}$'s password, so it chooses a random Z to run P_2
2. Sim submits pw_b via `LateTestPwd` to $\mathcal{F}_{\text{lePAKE}}$ and receives back K_b , where $K_b = K$ for $b = 0$ and random otherwise.
3. Sim chooses random $K_{1,1}, K_{1,2}$ to reply to $\mathcal{Z}$.
4. Sim obtains query $H_0(\text{sid}, \text{pw}_b, K_{1,1})$ and can now program it:
 - If Sim programs the hash to Z and $b = 1$, the test passes— $\mathcal{Z}$ can distinguish
 - If Sim programs the hash to a different Z' and $b = 0$, the test fails— $\mathcal{Z}$ can distinguish

References

1. Deployments of fancy cryptography (2024). <https://github.com/fancy-cryptography/fancy-cryptography>
2. Abdalla, M., Barbosa, M., Bradley, T., Jarecki, S., Katz, J., Xu, J.: Universally composable relaxed password authenticated key exchange. In: Micciancio, D., Ristenpart, T. (eds.) *CRYPTO 2020*. LNCS, vol. 12170, pp. 278–307. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-56784-2_10
3. Abdalla, M., Eisenhofer, T., Kiltz, E., Kunzweiler, S., Riepel, D.: Password-authenticated key exchange from group actions. In: Dodis, Y., Shrimpton, T. (eds.) *CRYPTO 2022, Part II*. LNCS, vol. 13508, pp. 699–728. Springer, Heidelberg (2022). https://doi.org/10.1007/978-3-031-15979-4_24
4. Abdalla, M., Haase, B., Hesse, J.: Security analysis of CPace. In: Tibouchi, M., Wang, H. (eds.) *ASIACRYPT 2021*. LNCS, vol. 13093, pp. 711–741. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-92068-5_24
5. Abdalla, M., Haase, B., Hesse, J.: CPace, a balanced composable PAKE. Internet Draft draft-irtf-cfrg-cpace-13, Internet engineering task force (2024). <https://datatracker.ietf.org/doc/draft-irtf-cfrg-cpace-13>, pp. 96
6. Abdalla, M., Pointcheval, D.: Simple password-based encrypted key exchange protocols. In: Menezes, A. (ed.) *CT-RSA 2005*. LNCS, vol. 3376, pp. 191–208. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-30574-3_14
7. Adrian, D., Beck, B., Benjamin, D., O'Brien, D.: Advancing our amazing bet on asymmetric cryptography (2024). <https://blog.chromium.org/2024/05/advancing-our-amazing-bet-on-asymmetric.html>
8. Alnahawi, N., Alperin-Sheriff, J., Apon, D., Wiesmaier, A.: NICE-PAKE: on the security of KEM-based PAKE constructions without ideal ciphers. *Cryptology ePrint Archive, Paper 2024/1957* (2024). <https://eprint.iacr.org/2024/1957>

9. Arriaga, A., Barbosa, M., Jarecki, S.: NoIC: PAKE from KEM without ideal ciphers. *Cryptology ePrint Archive*, Paper 2025/231 (2025). <https://eprint.iacr.org/2025/231>
10. Arriaga, A., Barbosa, M., Jarecki, S., Skrobot, M.: C'est très chic: a compact password-authenticated key exchange from lattice-based KEM. *Cryptology ePrint Archive*, Paper 2024/308 (2024). <https://eprint.iacr.org/2024/308>, <https://eprint.iacr.org/2024/308>
11. Barbosa, M., et al.: X-wing. *IACR Commun. Cryptol.* **1**(1) (2024). <https://doi.org/10.62056/a3qj89n4e>
12. Barbosa, M., Gellert, K., Hesse, J., Jarecki, S.: Bare PAKE: universally composable key exchange from just passwords. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024*, pp. 183–217. Springer Nature Switzerland, Cham (2024)
13. Beguinet, H., Chevalier, C., Pointcheval, D., Ricosset, T., Rossi, M.: GeT a CAKE: generic transformations from key encapsulation mechanisms to password authenticated key exchanges. In: Tibouchi, M., Wang, X. (eds.) *ACNS 23, Part II. LNCS*, vol. 13906, pp. 516–538. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-33491-7_19
14. Bellare, S., Merritt, M.: Encrypted key exchange: password-based protocols secure against dictionary attacks. In: *Proceedings 1992 IEEE Computer Society Symposium on Research in Security and Privacy*, pp. 72–84 (1992). <https://doi.org/10.1109/RISP.1992.213269>
15. Bernstein, D.J., et al.: KyberSlash: exploiting secret-dependent division timings in Kyber implementations. *Cryptology ePrint Archive*, Paper 2024/1049 (2024). <https://eprint.iacr.org/2024/1049>
16. Bernstein, D.J., Hamburg, M., Krasnova, A., Lange, T.: Elligator: elliptic-curve points indistinguishable from uniform random strings. In: Sadeghi, A.R., Gligor, V.D., Yung, M. (eds.) *ACM CCS 2013*, pp. 967–980. ACM Press (2013). <https://doi.org/10.1145/2508859.2516734>
17. Bindel, N., Hale, B.: A note on hybrid signature schemes. *Cryptology ePrint Archive*, Paper 2023/423 (2023). <https://eprint.iacr.org/2023/423>, <https://eprint.iacr.org/2023/423>
18. Bos, J.W., et al.: Frodo: Take off the ring! Practical, quantum-secure key exchange from LWE. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) *ACM CCS 2016*, pp. 1006–1018. ACM Press (2016). <https://doi.org/10.1145/2976749.2978425>
19. Bradley, T., Camenisch, J., Jarecki, S., Lehmann, A., Neven, G., Xu, J.: Password-Authenticated public-key encryption. In: Deng, R.H., Gauthier-Umaña, V., Ochoa, M., Yung, M. (eds.) *ACNS 2019. LNCS*, vol. 11464, pp. 442–462. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-21568-2_22
20. Canetti, R.: Universally composable security: a new paradigm for cryptographic protocols. In: *42nd FOCS*, pp. 136–145. IEEE Computer Society Press (2001). <https://doi.org/10.1109/SFCS.2001.959888>
21. Canetti, R., Halevi, S., Katz, J., Lindell, Y., MacKenzie, P.: Universally composable password-based key exchange. In: Cramer, R. (ed.) *EUROCRYPT 2005. LNCS*, vol. 3494, pp. 404–421. Springer, Heidelberg (2005). https://doi.org/10.1007/11426639_24
22. Canetti, R., Rabin, T.: Universal composition with joint state. In: Boneh, D. (ed.) *CRYPTO 2003. LNCS*, vol. 2729, pp. 265–281. Springer, Heidelberg (2003). https://doi.org/10.1007/978-3-540-45146-4_16

23. Ding, J., Alsayigh, S., Lancrenon, J., RV, S., Snook, M.: Provably secure password authenticated key exchange based on RLWE for the post-quantum world. In: Handschuh, H. (ed.) CT-RSA 2017. LNCS, vol. 10159, pp. 183–204. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-52153-4_11
24. Facebook: The labyrinth encrypted message storage protocol (2023). https://engineering.fb.com/wp-content/uploads/2023/12/TheLabyrinthEncryptedMessageStorageProtocol_12-6-2023.pdf
25. French cybersecurity agency: ANSSI views on the post-quantum cryptography transition (2022). <https://cyber.gouv.fr/sites/default/files/document/EN-Position.pdf>
26. Gajland, P., de Kock, B., Quaresma, M., Malavolta, G., Schwabe, P.: SWOOSH: efficient lattice-based non-interactive key exchange. In: 33rd USENIX Security Symposium (USENIX Security 24), pp. 487–504. USENIX Association, Philadelphia, PA (2024). <https://www.usenix.org/conference/usenixsecurity24/presentation/gajland>
27. Gentry, C., MacKenzie, P., Ramzan, Z.: A method for making password-based key exchange resilient to server compromise. In: Dwork, C. (ed.) CRYPTO 2006. LNCS, vol. 4117, pp. 142–159. Springer, Heidelberg (2006). https://doi.org/10.1007/11818175_9
28. German federal office for information security: BSI TR-02102-1, cryptographic mechanisms: recommendations and key lengths (2024). version 2024-01, <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf>
29. Gong, L.: Lower bounds on messages and rounds for network authentication protocols. In: Denning, D.E., Pyle, R., Ganesan, R., Sandhu, R.S., Ashby, V. (eds.) ACM CCS 93, pp. 26–37. ACM Press (1993). <https://doi.org/10.1145/168588.168592>
30. Guo, Q., Hlauschek, C., Johansson, T., Lahr, N., Nilsson, A., Schröder, R.L.: Don't reject this: key-recovery timing attacks due to rejection-sampling in HQC and BIKE. IACR TCHES **2022**(3), 223–263 (2022). <https://doi.org/10.46586/tches.v2022.i3.223-263>
31. Guo, Q., Johansson, T., Nilsson, A.: A Key-recovery timing attack on post-quantum primitives using the fujisaki-okamoto transformation and its application on FrodoKEM. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020. LNCS, vol. 12171, pp. 359–386. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-56880-1_13
32. Haase, B., Labrique, B.: AuCPace: efficient verifier-based PAKE protocol tailored for the IIoT. IACR TCHES **2019**(2), 1–48 (2019). <https://doi.org/10.13154/tches.v2019.i2.1-48>, <https://tches.iacr.org/index.php/TCHES/article/view/7384>
33. Hastad, J., Impagliazzo, R., Levin, L.A., Luby, M.: A pseudorandom generator from any one-way function. SIAM J. Comput. **28**(4), 1364–1396 (1999)
34. Hesse, J.: Separating symmetric and asymmetric password-authenticated key exchange. In: Galdi, C., Kolesnikov, V. (eds.) SCN 2020. LNCS, vol. 12238, pp. 579–599. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-57990-6_29
35. Hesse, J., Rosenberg, M.: PAKE combiners and efficient post-quantum instantiations. Cryptology ePrint Archive, Paper 2024/1621 (2024). <https://eprint.iacr.org/2024/1621>
36. Huang, S., Sim, R.Q., Chuengsatiansup, C., Guo, Q., Johansson, T.: Cache-timing attack against HQC. IACR TCHES **2023**(3), 136–163 (2023). <https://doi.org/10.46586/tches.v2023.i3.136-163>

37. Ishibashi, R., Yoneyama, K.: Compact password authenticated key exchange from group actions. In: Simpson, L., Bae, M.A.R. (eds.) ACISP 23. LNCS, vol. 13915, pp. 220–247. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-35486-1_11
38. Januzelli, J., Roy, L., Xu, J.: Under what conditions is encrypted key exchange actually secure? Cryptology ePrint Archive, Paper 2024/324 (2024). <https://eprint.iacr.org/2024/324>
39. Jarecki, S., Krawczyk, H., Xu, J.: OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. LNCS, vol. 10822, pp. 456–486. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78372-7_15
40. Katz, J., Rosenberg, M.: LATKE: a framework for constructing identity-binding PAKEs. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology - CRYPTO 2024, pp. 218–250. Springer Nature Switzerland, Cham (2024). <https://doi.org/10.1007/978-3-031-68379-4>
41. Katz, J., Vaikuntanathan, V.: Round-optimal password-based authenticated key exchange. In: Ishai, Y. (ed.) TCC 2011. LNCS, vol. 6597, pp. 293–310. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-19571-6_18
42. Ladd, W.: SPAKE2, a password-authenticated key exchange. Request for comments RFC 9382, internet engineering task force (2023). <https://doi.org/10.17487/RFC9382>, <https://datatracker.ietf.org/doc/rfc9382>, pp. 17
43. Langley, A., Hamburg, M., Turner, S.: Elliptic Curves for Security. RFC 7748 (Informational) (2016). <https://doi.org/10.17487/RFC7748>, <https://www.rfc-editor.org/rfc/rfc7748.txt>
44. Lindner, R., Peikert, C.: Better key sizes (and attacks) for LWE-based encryption. In: Kiayias, A. (ed.) CT-RSA 2011. LNCS, vol. 6558, pp. 319–339. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-19074-2_21
45. Lyu, Y., Liu, S.: Hybrid password authentication key exchange in the UC framework. Cryptology ePrint Archive, Paper 2024/1630 (2024). <https://eprint.iacr.org/2024/1630>
46. Lyu, Y., Liu, S., Han, S.: Universal composable password authenticated key exchange for the post-quantum world. In: Joye, M., Leander, G. (eds.) Advances in Cryptology - EUROCRYPT 2024, pp. 120–150. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-58754-2_5
47. Lyu, Y., Liu, S., Han, S.: Efficient asymmetric PAKE compiler from KEM and AE. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024, pp. 34–65. Springer Nature Singapore, Singapore (2025). https://doi.org/10.1007/978-981-96-0935-2_2
48. McQuoid, I., Rosulek, M., Roy, L.: Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020, pp. 425–442. ACM Press (2020). <https://doi.org/10.1145/3372297.3417870>
49. Pan, J., Zeng, R.: A generic construction of tightly secure password-based authenticated key exchange. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part VIII. LNCS, vol. 14445, pp. 143–175. Springer, Heidelberg (2023). https://doi.org/10.1007/978-981-99-8742-9_5
50. Pointcheval, D., Wang, G.: VTBPEKE: verifier-based two-basis password exponential key exchange. In: Karri, R., Sinanoglu, O., Sadeghi, A.R., Yi, X. (eds.) ASIACCS 17, pp. 301–312. ACM Press (2017)

51. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC, pp. 84–93. ACM Press (2005). <https://doi.org/10.1145/1060590.1060603>
52. Ren, P., Gu, X.: Practical post-quantum password-authenticated key exchange based-on module-lattice. In: Park, J.H., Seo, S.H. (eds.) Information Security and Cryptology - ICISC 2021, pp. 137–156. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-08896-4_7
53. Santos, B.F.D., Gu, Y., Jarecki, S.: Randomized half-ideal cipher on groups with applications to UC (a)PAKE. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 128–156. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-30589-4_5
54. Shoup, V.: Security analysis of *SPAKE2+*. In: Pass, R., Pietrzak, K. (eds.) TCC 2020. LNCS, vol. 12552, pp. 31–60. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64381-2_2
55. Westerbaan, B.: The state of the post-quantum Internet (2024). <https://blog.cloudflare.com/pq-2024>, the Cloudflare Blog